

Lab 2 Exam

准备工作：创建并切换到 lab2-exam 分支

1. 基于已完成的 lab2 提交自动初始化 lab2-exam 分支
2. 在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab2-exam
```

初始化的 lab2-exam 分支基于课下完成的 lab2 分支，并且在 tests 目录下添加了 lab2_list_insert_at 样例测试目录。

题目背景 & 题目描述

在操作系统内核中，双向链表（如 <queue.h> 中定义的 LIST 系列宏）被广泛应用于物理内存页管理、进程控制块队列等重要数据结构的维护。现有的链表宏提供了头部插入（LIST_INSERT_HEAD）、在特定元素前后插入（LIST_INSERT_AFTER / LIST_INSERT_BEFORE）等功能。

在某些特定的调度或分配场景下，我们需要将一个元素插入到链表的**特定索引位置**。在本题中，你需要实现一个新的 C 语言宏 LIST_INSERT_AT，**将指定的元素插入到给定链表的第 index 个位置**。

```
#define LIST_INSERT_AT(type, head, elm, field, index)
```

该宏各参数的意义与具体功能描述如下：

通过调用此宏，将元素 elm 插入到链表 head 的第 index 个位置（索引从 0 开始）。

- type：链表节点对应的结构体类型名（例如 Page）。
- head：指向链表头部的指针。
- elm：待插入的元素指针。
- field：结构体中用于链接的 LIST_ENTRY 字段名（例如 pp_link）。
- index：目标插入位置的索引。
- 如果 index == 0，则将 elm 插入到链表的最头部，使其成为链表新的首个元素（即第 0 个元素）。
- 如果 index > 0，则将其插入到原链表第 index - 1 个元素的后面。
- 测试样例保证传入的 index 大于等于 0 且小于等于当前链表的长度（当 index 等于链表长度时，意味着将元素追加到链表最尾部）。
- 测试样例保证传入的 index 表达式不存在副作用。

题目要求

在 include/queue.h 中添加 LIST_INSERT_AT 宏的定义。

提示

1. 本题**要求使用宏实现**，不能使用普通的 C 函数。在评测中，测试代码可能会将不同的类型名（如 `Page`）作为参数传入，普通函数无法接收类型名作为参数。
2. 可以组合使用已有的链表宏。例如：
 - 当 `index == 0` 时，可以直接调用 `LIST_INSERT_HEAD`。
 - 当 `index > 0` 时，可以使用 `LIST_FOREACH` 宏配合自定义的计数器遍历链表，找到对应的节点后使用 `LIST_INSERT_AFTER` 插入。
4. **注意 C 语言宏的安全编程规范**：
 - 宏的外部应使用 `do { ... } while (0)` 包裹以保证语法的安全性。
 - 宏内部使用传入的参数时，建议加上括号（如 `(index)`），防止表达式优先级导致的逻辑错误。
 - 宏内部声明临时变量时（如计数器变量或迭代指针），建议使用带有下划线前缀的命名（如 `_cnt`，`_var`），以防止与外部作用域的变量名发生冲突。
5. 评测会借助课下已给出的 `include/queue.h` 中的相关链表宏，请保证没有对官方代码进行魔改。

样例输出 & 本地测试

对于以下样例：

```
#include <mmu.h>
#include <pmap.h>
#include <print.h>

void test_list_insert_at() {
    struct Page_list test_list;
    LIST_INIT(&test_list);

    struct Page *pages = (struct Page *)alloc(5 * sizeof(struct Page), PAGE_SIZE,
1);

    LIST_INSERT_AT(Page, &test_list, &pages[0], pp_link, 0);
    LIST_INSERT_AT(Page, &test_list, &pages[2], pp_link, 1);
    LIST_INSERT_AT(Page, &test_list, &pages[1], pp_link, 1);
    LIST_INSERT_AT(Page, &test_list, &pages[3], pp_link, 0);
    LIST_INSERT_AT(Page, &test_list, &pages[4], pp_link, 4);

    struct Page *answer[] = {
        &pages[3],
        &pages[0],
        &pages[1],
        &pages[2],
        &pages[4]
    };

    struct Page *p = LIST_FIRST(&test_list);
    int j = 0;

    while (p != NULL) {
        assert(p == answer[j++]);
        p = LIST_NEXT(p, pp_link);
    }
}
```

```
}
assert(j == 5);

printf("test succeeded!\n");
}
```

其应当输出：

```
test succeeded!
```

你可以使用：

`make test lab=2_list_insert_at && make run` 在本地测试上述样例（调试模式）

`MOS_PROFILE=release make test lab=2_list_insert_at && make run` 在本地测试上述样例（开启优化）

或者在 `init/init.c` 的 `mips_init` 函数中自行编写测试代码并调用 `test_list_insert_at()` 使用 `make && make run` 测试。

提交评测 & 评测标准

请在开发机中执行下列命令后，在[课程网站上提交评测](#)。

```
$ cd ~/学号/
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push
```

测试点说明及分数分布如下：

测试点序号	评测说明	分值
1	与样例相同	10 分
2	仅测试基于空链表及非空链表的头部插入 (<code>index == 0</code>)	20 分
3	仅测试在链表中间位置的插入 (<code>0 < index < length</code>)	25 分
4	仅测试在链表末尾追加元素 (<code>index == length</code>)	25 分
5	综合评测	20 分